

STUDENT HANDBOOK

---

# END-TO-END DATA ANALYTICS PROJECT HANDBOOK

*(SQL + Python Analytics Project)*

A Complete Project-Based Learning Guide for  
**B.Tech Computer Science Students**

Department of Computer Science & Engineering  
Final Year Project — Data Analytics Track  
Academic Year 2026

## Table of Contents

---

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Table of Contents .....                                                    | 2  |
| Chapter 1: Project Overview .....                                          | 7  |
| 1.1 What Is an End-to-End Data Analytics Project? .....                    | 7  |
| 1.2 Why Companies Expect the Complete Lifecycle .....                      | 7  |
| 1.3 Database Development vs. Data Analytics vs. Business Intelligence..... | 7  |
| 1.4 How SQL and Python Complement Each Other .....                         | 8  |
| 1.5 Objectives of This Project.....                                        | 9  |
| 1.6 Learning Outcomes .....                                                | 9  |
| Chapter Summary .....                                                      | 9  |
| Key Takeaways .....                                                        | 10 |
| Next Steps .....                                                           | 10 |
| Chapter 2: Industry Workflow .....                                         | 11 |
| 2.1 The Professional Analytics Workflow .....                              | 11 |
| 2.2 Understanding Each Stage .....                                         | 12 |
| 2.3 Why This Workflow Matters for Your Project.....                        | 13 |
| Chapter Summary .....                                                      | 14 |
| Key Takeaways .....                                                        | 14 |
| Next Steps .....                                                           | 14 |
| Chapter 3: Project Roadmap.....                                            | 15 |
| 3.1 The Complete Project Roadmap .....                                     | 15 |
| 3.2 Phase-by-Phase Explanation and Expected Outputs .....                  | 16 |
| 3.3 Reading the Roadmap Correctly .....                                    | 17 |
| Chapter Summary .....                                                      | 17 |
| Key Takeaways .....                                                        | 17 |
| Next Steps .....                                                           | 18 |
| Chapter 4: Extending the SQL Project.....                                  | 19 |
| 4.1 One Project, Two Phases.....                                           | 19 |
| 4.2 Worked Example: Hospital Management Database .....                     | 19 |
| 4.3 How This Applies Across Domains .....                                  | 20 |
| 4.4 SQL as the Data Source for Python .....                                | 21 |
| Chapter Summary .....                                                      | 21 |
| Key Takeaways .....                                                        | 21 |
| Next Steps .....                                                           | 21 |

|                                                     |    |
|-----------------------------------------------------|----|
| Chapter 5: Technology Stack.....                    | 22 |
| 5.1 MySQL .....                                     | 22 |
| 5.2 MySQL Workbench .....                           | 22 |
| 5.3 Python .....                                    | 22 |
| 5.4 Anaconda .....                                  | 22 |
| 5.5 Jupyter Notebook.....                           | 23 |
| 5.6 Pandas .....                                    | 23 |
| 5.7 NumPy .....                                     | 23 |
| 5.8 Matplotlib .....                                | 23 |
| 5.9 CSV Files .....                                 | 24 |
| 5.10 Git.....                                       | 24 |
| 5.11 GitHub .....                                   | 24 |
| 5.12 Stack Summary.....                             | 24 |
| Chapter Summary .....                               | 25 |
| Key Takeaways .....                                 | 25 |
| Next Steps .....                                    | 25 |
| Chapter 6: Connecting Python with SQL.....          | 26 |
| 6.1 Architecture Overview .....                     | 26 |
| 6.2 Installing the Connector.....                   | 26 |
| 6.3 Establishing the Connection.....                | 27 |
| 6.4 Reading an Entire Table .....                   | 27 |
| 6.5 Reading a Custom SQL Query .....                | 27 |
| 6.6 pandas.read_sql() in Detail .....               | 28 |
| 6.7 Closing the Connection .....                    | 28 |
| 6.8 Common Errors and Troubleshooting.....          | 28 |
| Best Practices .....                                | 28 |
| Chapter Summary .....                               | 29 |
| Key Takeaways .....                                 | 29 |
| Next Steps .....                                    | 29 |
| Chapter 7: Python Concepts Used in the Project..... | 30 |
| 7.1 Core Python Fundamentals.....                   | 30 |
| 7.2 NumPy in This Project .....                     | 31 |
| 7.3 Pandas in This Project .....                    | 31 |
| 7.4 Statistics Used in This Project .....           | 31 |
| 7.5 Matplotlib in This Project.....                 | 31 |

|                                              |    |
|----------------------------------------------|----|
| 7.6 Business KPI Calculations .....          | 32 |
| Chapter Summary .....                        | 32 |
| Key Takeaways .....                          | 33 |
| Next Steps .....                             | 33 |
| Chapter 8: Complete Analytics Workflow.....  | 34 |
| 8.1 Data Extraction.....                     | 34 |
| 8.2 Data Cleaning .....                      | 34 |
| 8.3 Data Validation.....                     | 35 |
| 8.4 Feature Engineering.....                 | 35 |
| 8.5 Exploratory Data Analysis (EDA) .....    | 35 |
| 8.6 KPI Calculation .....                    | 35 |
| 8.7 Visualization .....                      | 36 |
| 8.8 Business Insights .....                  | 36 |
| 8.9 Recommendations .....                    | 36 |
| Chapter Summary .....                        | 37 |
| Key Takeaways .....                          | 37 |
| Next Steps .....                             | 37 |
| Chapter 9: Project Directory Structure ..... | 38 |
| 9.1 Recommended Folder Structure.....        | 38 |
| 9.2 What Each Folder Contains.....           | 38 |
| 9.3 Naming Conventions.....                  | 39 |
| Chapter Summary .....                        | 39 |
| Key Takeaways.....                           | 39 |
| Next Steps .....                             | 40 |
| Chapter 10: GitHub Repository Structure..... | 41 |
| 10.1 Repository Naming.....                  | 41 |
| 10.2 Folder Organisation on GitHub .....     | 41 |
| 10.3 Images and Screenshots .....            | 41 |
| 10.4 Recommended Commit Messages .....       | 41 |
| 10.5 Branching (Optional).....               | 42 |
| 10.6 GitHub Best Practices.....              | 42 |
| 10.7 Professional README Template .....      | 42 |
| Chapter Summary .....                        | 43 |
| Key Takeaways .....                          | 43 |
| Next Steps .....                             | 43 |

|                                                      |    |
|------------------------------------------------------|----|
| Chapter 11: Business Insights .....                  | 44 |
| 11.1 Why Charts Alone Are Not Enough.....            | 44 |
| 11.2 The Insight Framework .....                     | 44 |
| 11.3 Worked Examples Across Domains.....             | 45 |
| 11.4 Writing Insights: Do's and Don'ts.....          | 45 |
| Chapter Summary .....                                | 46 |
| Key Takeaways .....                                  | 46 |
| Next Steps .....                                     | 46 |
| Chapter 12: Final Deliverables .....                 | 47 |
| 12.1 SQL Phase Deliverables .....                    | 47 |
| 12.2 Python Analytics Phase Deliverables .....       | 47 |
| 12.3 Documentation & Presentation Deliverables ..... | 47 |
| Chapter Summary .....                                | 48 |
| Key Takeaways .....                                  | 48 |
| Next Steps .....                                     | 48 |
| Chapter 13: Best Practices .....                     | 49 |
| 13.1 Professional Coding Standards .....             | 49 |
| 13.2 Naming Conventions.....                         | 49 |
| 13.3 Documentation .....                             | 49 |
| 13.4 Folder Organisation.....                        | 49 |
| 13.5 SQL Formatting .....                            | 49 |
| 13.6 Python Formatting.....                          | 50 |
| 13.7 Comments.....                                   | 50 |
| 13.8 Exception Handling .....                        | 50 |
| 13.9 Version Control .....                           | 50 |
| 13.10 Code Reusability & Modular Programming .....   | 50 |
| 13.11 Project Backup .....                           | 51 |
| 13.12 Professional Presentation Tips.....            | 51 |
| Chapter Summary .....                                | 51 |
| Key Takeaways .....                                  | 51 |
| Next Steps .....                                     | 51 |
| Chapter 14: Common Mistakes .....                    | 52 |
| Chapter Summary .....                                | 53 |
| Key Takeaways .....                                  | 53 |
| Next Steps .....                                     | 53 |

|                                              |    |
|----------------------------------------------|----|
| Chapter 15: Final Submission Checklist ..... | 54 |
| 15.1 Database & SQL.....                     | 54 |
| 15.2 Python & Analysis .....                 | 54 |
| 15.3 Documentation & Repository .....        | 54 |
| 15.4 Presentation & Demonstration.....       | 54 |

© Shalinee Priya

# Chapter 1: Project Overview

---

Welcome to the End-to-End Data Analytics Project. This handbook extends the SQL database project you have already built into a complete, industry-style analytics solution using Python. You are not starting a new project — you are completing the one you started.

## 1.1 What Is an End-to-End Data Analytics Project?

An End-to-End Data Analytics Project is a single, connected body of work that carries a business problem all the way from a raw idea to a decision-ready recommendation. It does not stop at building a database, and it does not stop at drawing a chart. It begins with a business question, moves through database design and SQL development, continues into Python-based data extraction and analysis, and finishes with visualizations, insights, and recommendations that a business could act on.

The word "end-to-end" is important. In academic settings, students often treat the database phase and the analytics phase as two disconnected assignments. In the industry, they are two halves of one deliverable. A recruiter or a project reviewer will always ask the same question: "Show me the complete journey — from the problem to the decision." This handbook is designed to help you answer that question with confidence.

## 1.2 Why Companies Expect the Complete Lifecycle

Organisations do not hire people who can only write SQL, or people who can only plot charts. They hire people who can take ownership of a problem end-to-end: understand what the business needs, design a system to store the data correctly, extract and clean it, analyse it, and communicate a decision-ready recommendation. This is true whether the final job title is Data Analyst, Data Engineer, Business Analyst, or Software Engineer.

- Employers want proof that you can reason about a business problem, not just execute a technical task.
- Real projects rarely arrive as clean, ready-to-analyse data — you are expected to build or repair the pipeline yourself.
- The ability to move fluently between SQL and Python is one of the most in-demand combinations in data roles today.
- A candidate who can present a complete analytics story stands out far more than one who can only demonstrate isolated skills.

## 1.3 Database Development vs. Data Analytics vs. Business Intelligence

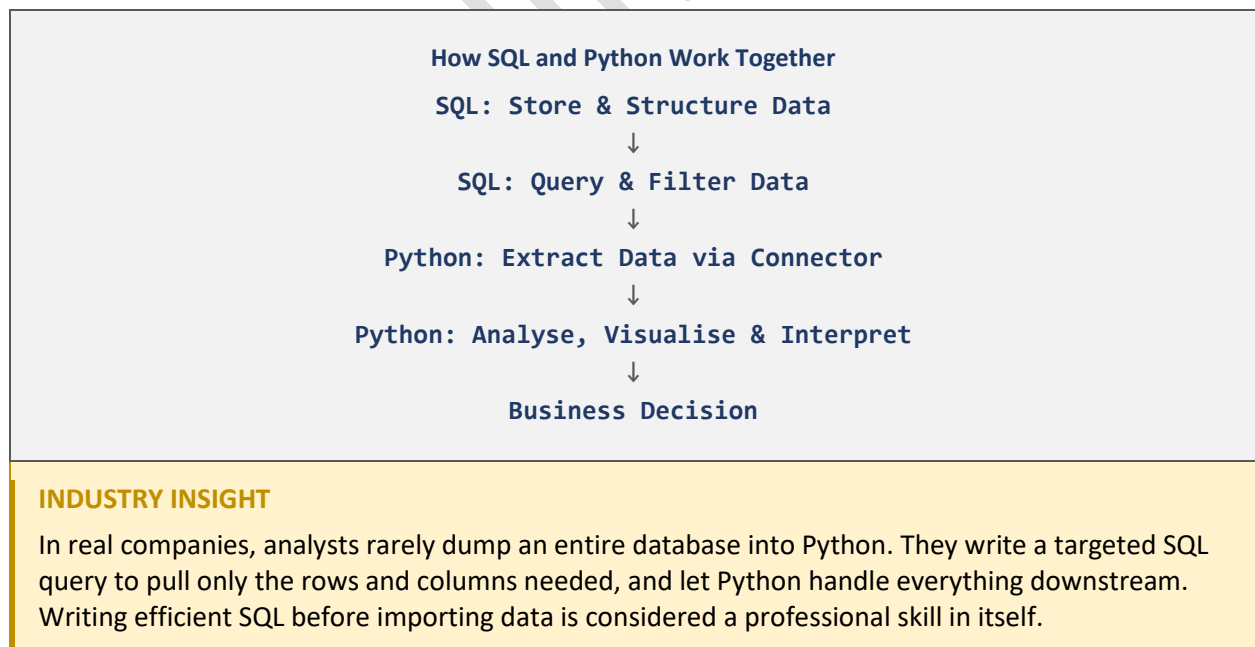
These three terms are frequently used interchangeably by students, but they represent distinct disciplines with different goals, tools, and outputs. Understanding the difference will help you describe your own project accurately in interviews and reports.

| Aspect        | Database Development                   | Data Analytics                            | Business Intelligence                     |
|---------------|----------------------------------------|-------------------------------------------|-------------------------------------------|
| Primary Goal  | Store data accurately and efficiently  | Explore data to find patterns and answers | Deliver ongoing, decision-ready reporting |
| Core Question | How should the data be structured?     | What does the data tell us?               | How do we monitor this continuously?      |
| Typical Tools | MySQL, ER modelling, SQL               | Python, Pandas, NumPy, Statistics         | Power BI, Tableau, Dashboards             |
| Output        | Normalised schema, tables, constraints | Insights, KPIs, charts, reports           | Live dashboards, scorecards               |
| Time Horizon  | Built once, maintained                 | Project-based or periodic                 | Continuous / real-time                    |

Your project touches all three disciplines. The SQL phase covered Database Development. This handbook covers Data Analytics. Several of the techniques you will learn — KPIs, recurring reports, and visual dashboards — are the foundation of Business Intelligence, even though you will build them as static Python outputs rather than live dashboards.

## 1.4 How SQL and Python Complement Each Other

SQL and Python are not competitors — they are partners that are strong in different places. SQL is the best tool in the world for storing, structuring, and reliably retrieving data. Python is the best tool for flexible analysis, statistics, automation, and visualization. A professional analyst uses SQL to get clean, well-shaped data out of a database, and Python to turn that data into insight.





## 1.5 Objectives of This Project

- Extend your existing SQL database into a working analytics pipeline using Python.
- Practise real-world data extraction, cleaning, and validation techniques.
- Apply Python libraries such as Pandas, NumPy, and Matplotlib to answer business questions.
- Translate raw analysis into clear, decision-ready business insights and recommendations.
- Produce a portfolio-quality project with professional documentation, folder structure, and a GitHub repository.
- Build the communication skill of presenting technical work to a non-technical audience.

## 1.6 Learning Outcomes

By the end of this project, you will be able to:

- Connect a Python application to a MySQL database and retrieve data reliably.
- Clean and validate real-world, imperfect data using Pandas.
- Perform exploratory data analysis (EDA) to understand structure, trends, and anomalies.
- Calculate business Key Performance Indicators (KPIs) relevant to your chosen domain.
- Design and generate meaningful charts using Matplotlib.
- Write professional business insights that connect data to decisions.
- Organise, document, and publish a complete analytics project on GitHub.

### TIP

Keep a short project journal from Day 1 — one line per day describing what you did. It will save you hours when writing your final report and preparing for your presentation.

---

## Chapter Summary

- An end-to-end analytics project connects a business problem to a database, and a database to a decision.
- Database Development, Data Analytics, and Business Intelligence are related but distinct disciplines.
- SQL and Python are complementary tools, not competing ones — SQL structures data, Python analyses it.

## Key Takeaways

- You are extending your SQL project, not starting a new one.
- Employers value the complete lifecycle far more than isolated technical skills.
- Clear learning outcomes exist for this project and will be revisited in the final checklist.

## Next Steps

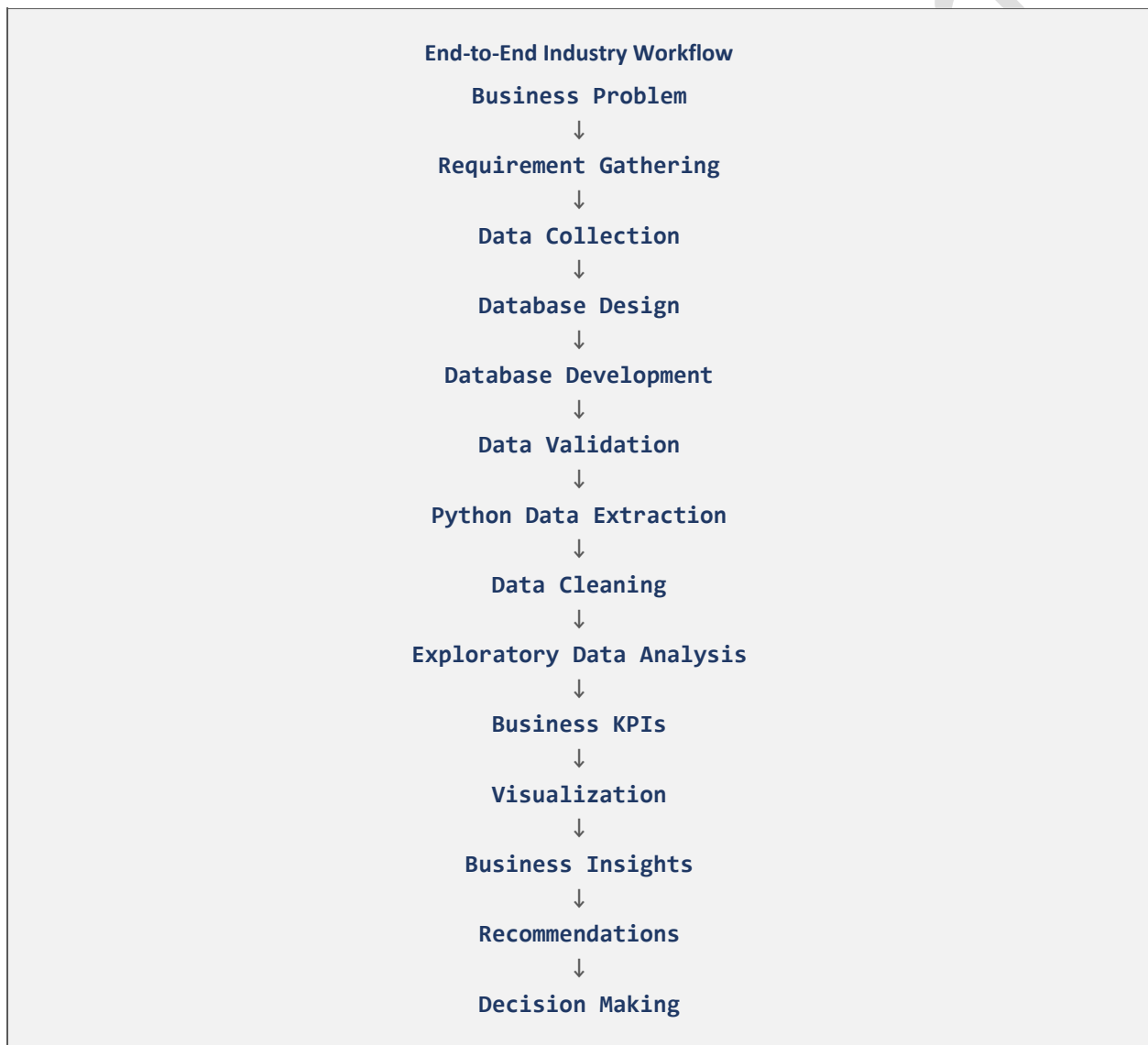
- Read Chapter 2 to understand how real companies execute analytics projects.
- Revisit your SQL project and confirm your database schema is finalised before proceeding.

© Shalinee Priya

## Chapter 2: Industry Workflow

Before you touch a single line of code, it is important to understand how real companies approach a data analytics engagement from start to finish. This chapter focuses purely on the business workflow — the sequence of stages a professional team follows regardless of the tools involved. Your student roadmap, covered in Chapter 3, is a practical adaptation of this same workflow.

### 2.1 The Professional Analytics Workflow



## 2.2 Understanding Each Stage

### Business Problem

Every analytics engagement starts with a real pain point: falling sales, high patient wait times, delayed deliveries, fraud losses, or churn. Without a clearly defined problem, no amount of data or code produces value. Companies invest significant time here because a wrongly framed problem leads to wasted analysis.

### Requirement Gathering

Analysts meet stakeholders to understand exactly what questions need answering, who will use the results, and what decisions depend on them. This stage defines scope and prevents the team from analysing data that nobody needs.

### Data Collection

Teams identify what data already exists, what must be newly captured, and where it lives — internal systems, third-party sources, logs, or manual records. Data collection determines what questions can realistically be answered.

### Database Design

Before any table is created, the data is modelled: entities, relationships, and structure are planned using ER diagrams and normalisation rules. Good design here prevents duplicate data, inconsistent values, and slow queries later.

### Database Development

The designed model is implemented: tables, constraints, relationships, views, stored procedures, and triggers are built. This is the phase you have already completed in your SQL project.

### Data Validation

Before analysis begins, the data is checked for correctness — are foreign keys consistent, are there missing mandatory fields, do sample records make logical sense? Skipping validation is one of the most common causes of wrong business conclusions.

### Python Data Extraction

A Python application connects to the database and pulls the specific data needed for analysis, using SQL queries executed from Python. This is the bridge between the two halves of your project.

### Data Cleaning

Even validated database data is rarely analysis-ready. Analysts handle missing values, incorrect data types, duplicate rows, and inconsistent formatting using Pandas before any analysis begins.

## Exploratory Data Analysis (EDA)

Analysts examine the data's shape, distributions, trends, and relationships without a fixed hypothesis, to build an intuitive understanding of what the data contains and where the interesting patterns might be.

## Business KPIs

Raw statistics are converted into metrics the business actually cares about — revenue growth, average order value, patient readmission rate, on-time delivery percentage — depending on the domain.

## Visualization

Charts are built to make patterns and KPIs immediately understandable to a non-technical audience. The right chart type is chosen deliberately, not by default.

## Business Insights

Numbers and charts are translated into written statements that explain what is happening, why it might be happening, and what impact it has on the business.

## Recommendations

Insights are converted into specific, actionable suggestions — what should the business do differently based on what was found.

## Decision Making

Ultimately, stakeholders use the recommendations to make real decisions: adjusting pricing, reallocating staff, changing a process, or investing in a new area. This is the entire reason the workflow exists.

### INDUSTRY INSIGHT

Notice that coding appears in only two of the fourteen stages above. In the industry, technical execution is a small part of a much larger process built on understanding the problem and communicating the outcome. Students who focus only on code often produce technically correct but commercially uninteresting projects.

## 2.3 Why This Workflow Matters for Your Project

- It gives you a mental map of where each activity in your project fits into the bigger picture.
- It explains why insights and recommendations (Chapter 11) carry as much weight as your code.
- It mirrors exactly how a Data Analyst's or Data Engineer's first few months on the job will look.
- It sets the expectation that every chart or query you produce should trace back to a business problem.

### NOTE

This chapter deliberately does not mention student submissions, GitHub, or grading. Chapter 3 translates this same workflow into a concrete, submission-ready roadmap for your project.

---

## Chapter Summary

- The industry analytics workflow has fourteen connected stages, from business problem to decision making.
- Each stage exists to reduce risk — of solving the wrong problem, of analysing bad data, or of producing insights nobody can use.
- Coding is a small part of the overall workflow; framing, validation, and communication carry equal weight.

## Key Takeaways

- Data Validation and Data Cleaning are distinct steps — one checks the database, the other prepares data in Python.
- KPIs and visualizations only matter if they are tied back to the original business problem.
- Recommendations, not charts, are what actually drive decision making.

## Next Steps

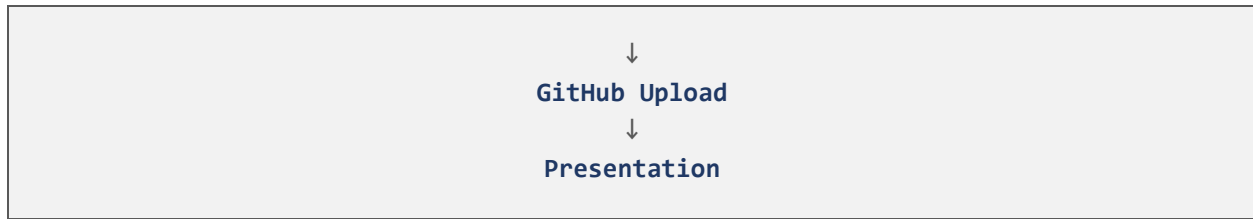
- Proceed to Chapter 3 to see this workflow translated into a step-by-step project roadmap.
- Re-read your SQL project's business problem statement — you will reuse it throughout this handbook.

## Chapter 3: Project Roadmap

This chapter converts the industry workflow from Chapter 2 into a concrete, step-by-step roadmap for completing your End-to-End Data Analytics Project. Follow these phases in order — each phase produces an output that the next phase depends on.

### 3.1 The Complete Project Roadmap





### 3.2 Phase-by-Phase Explanation and Expected Outputs

| Phase                | What You Do                                                      | Expected Output                             |
|----------------------|------------------------------------------------------------------|---------------------------------------------|
| Choose Domain        | Select a real-world domain (hospital, e-commerce, banking, etc.) | Confirmed project domain                    |
| Problem Statement    | Write a one-paragraph business problem your project solves       | Problem statement document                  |
| Requirement Analysis | List the entities, data points, and questions to be answered     | Requirement document                        |
| ER Diagram           | Model entities and relationships visually                        | ER diagram (already completed in SQL phase) |
| Database Design      | Convert ER diagram into a relational schema                      | Schema design (already completed)           |
| Normalization        | Remove redundancy up to 3NF                                      | Normalised schema (already completed)       |
| Create Tables        | Implement schema in MySQL with constraints                       | Working database (already completed)        |
| Insert Data          | Populate tables with realistic sample data                       | Populated database (already completed)      |
| Advanced SQL         | Views, stored procedures, triggers, window functions             | SQL script files (already completed)        |
| Connect Python       | Establish a Python-to-MySQL connection                           | Working DB connection script                |
| Load Data            | Pull required tables/queries into Pandas DataFrames              | Loaded DataFrames                           |
| Data Cleaning        | Handle missing values, duplicates, and type issues               | Clean, analysis-ready DataFrames            |
| EDA                  | Explore structure, trends, and distributions                     | EDA notebook/script with findings           |
| KPIs                 | Calculate domain-specific business metrics                       | KPI values and summary table                |
| Charts               | Visualise KPIs and trends with Matplotlib                        | Saved chart images                          |
| Business Insights    | Write Observation → Reason → Impact → Recommendation             | Insights document                           |
| Documentation        | Write README and project report                                  | README.md and report                        |



| Phase         | What You Do                            | Expected Output                   |
|---------------|----------------------------------------|-----------------------------------|
| GitHub Upload | Push the complete, organised project   | Public/shared GitHub repository   |
| Presentation  | Prepare and rehearse your project demo | Slide deck and live/recorded demo |

### 3.3 Reading the Roadmap Correctly

Notice that the first nine phases are the SQL work you have already completed or are completing. This handbook begins in earnest at "Connect Python" — but it revisits the earlier phases briefly wherever your SQL decisions affect your Python analysis (for example, table naming, data types, and relationships).

- Treat each phase as a checkpoint, not a race — do not start Python work with an unfinished or unvalidated database.
- Each phase output becomes an input to the next; skipping outputs (e.g., no ER diagram) causes rework later.
- The roadmap is linear for planning purposes, but in practice you will revisit earlier phases (e.g., re-cleaning data after new EDA findings).

#### TIP

Print or pin this roadmap where you can see it. At any point in the project, you should be able to point to exactly which phase you are in and what output you owe by the end of it.

#### NOTE

If your SQL phase is not yet fully complete, finish Advanced SQL (views, procedures, triggers, window functions) before starting the Python phases. Python analysis on an incomplete or unstable schema leads to repeated rework.

## Chapter Summary

- The roadmap has nineteen phases, moving from domain selection through to final presentation.
- The first nine phases are your existing SQL project; Python work begins at 'Connect Python'.
- Each phase has a defined, checkable output — use these as your personal milestone checklist.

## Key Takeaways

- This is one continuous project, not two separate ones — the roadmap makes that continuity explicit.
- A stable, validated database is a prerequisite for reliable Python analysis.
- Documentation and presentation are formal phases, not afterthoughts.

## Next Steps

- Confirm which phase you are currently in using the table in Section 3.2.
- Move to Chapter 4 to see exactly how your specific SQL domain extends into analytics.

© Shalinee Priya

## Chapter 4: Extending the SQL Project

This is the most important mindset shift in the entire handbook: you are not building a second, separate project. You are extending the SQL database you already designed into a complete analytics solution. Every table, view, and relationship you built becomes the raw material for the analysis you are about to perform.

### 4.1 One Project, Two Phases

Think of your SQL work as the foundation of a building and your Python work as everything built on top of it — the rooms, the wiring, the finished interior. Without the foundation, there is nothing to build on. Without the interior, the foundation alone is not livable. Both phases together make one complete structure: your End-to-End Data Analytics Project.

### 4.2 Worked Example: Hospital Management Database

Suppose your SQL project is a Hospital Management System with tables for patients, doctors, appointments, departments, billing, and medicines. Chapter 2's workflow applies directly:



- Patient Analytics: admission trends, average length of stay, age and gender distribution, readmission patterns.
- Revenue Analytics: total billing by month, revenue by department, outstanding payments, insurance vs. self-pay split.
- Doctor Performance: patients handled per doctor, average consultation time, specialisation-wise demand.
- Department Analysis: department-wise patient load, resource utilisation, peak admission times.
- Medicine Analysis: most prescribed medicines, stock consumption trends, expiry risk analysis.

Every one of these analyses is simply a Python script running a SQL query against tables you have already created, followed by cleaning, KPI calculation, and visualization. Nothing about the database changes — only how you extract insight from it.

### 4.3 How This Applies Across Domains

The same extension pattern applies regardless of which domain you chose for your SQL project. The table below shows how a handful of common student domains extend naturally into analytics themes.

| Domain               | Example Analytics Extensions                                                     |
|----------------------|----------------------------------------------------------------------------------|
| E-commerce           | Sales trends, top products, customer segmentation, cart abandonment, return rate |
| Banking              | Transaction patterns, loan default risk, account growth, branch performance      |
| IPL / Sports         | Player performance, team win ratios, venue analysis, season-over-season trends   |
| Library              | Book circulation trends, overdue analysis, popular genres, member activity       |
| Inventory            | Stock turnover, reorder patterns, slow-moving items, supplier performance        |
| Hotel                | Occupancy rate, average booking value, seasonal demand, cancellation trends      |
| Employee Management  | Attrition trends, department headcount, salary distribution, leave patterns      |
| Food Delivery        | Delivery time analysis, order volume trends, restaurant performance, peak hours  |
| Movie Streaming      | Watch-time trends, genre popularity, subscriber churn, content performance       |
| University Placement | Placement rate by branch, package trends, company-wise hiring, skill gaps        |
| Airline              | On-time performance, route profitability, seat occupancy, booking lead time      |
| Smart City           | Traffic patterns, utility consumption, complaint resolution time, sensor trends  |
| Fraud Detection      | Anomaly patterns, transaction risk scoring, flagged-account trends               |
| Construction         | Project cost overruns, timeline delays, material consumption, vendor performance |
| API Management       | Request volume trends, error rate analysis, latency patterns, usage by client    |

#### INDUSTRY INSIGHT

Notice a pattern: almost every domain's analytics extensions fall into the same four buckets — volume/trend analysis, performance analysis, financial analysis, and risk/quality analysis. Once you recognise these buckets, you can generate meaningful analysis questions for any domain in minutes.

## 4.4 SQL as the Data Source for Python

From this point forward, think of your SQL database as a data source, in exactly the same way a professional analyst would think of a company's production database. Python does not replace SQL — it consumes SQL's output. Your queries decide what raw material Python receives; your cleaning and analysis decide what is made from it.

- Well-written SQL queries (joins, filters, aggregations) reduce the cleaning work needed in Python.
- Views you built in your SQL phase can be queried directly from Python exactly like tables.
- Stored procedures can be called from Python where business logic should stay inside the database.
- Window functions used in SQL (e.g., ranking, running totals) can save you significant Pandas code later.

### NOTE

If you find yourself wanting to write a new SQL feature (a new view, or a slightly different query) to support your analysis, that is expected and encouraged — it is still part of the same project, not a new one.

---

## Chapter Summary

- Your Python analytics work is an extension of your SQL project, never a separate one.
- The same six-step extension pattern (data → domain-specific analytics areas) applies across all domains.
- Your SQL database is now your project's official data source.

## Key Takeaways

- Well-designed SQL queries reduce downstream cleaning effort in Python.
- Views, stored procedures, and window functions built in SQL can be reused directly in the Python phase.
- Every domain's analytics needs fall into recognisable categories: volume, performance, financial, and risk/quality.

## Next Steps

- Identify 4–6 analytics questions specific to your own domain, modelled on Section 4.3.
- Proceed to Chapter 5 to review the technology stack you will use to answer them.

## Chapter 5: Technology Stack

---

This chapter introduces every tool used in your End-to-End Data Analytics Project. For each one, you will find its purpose, how it is used in the industry, and why it belongs in this specific project.

### 5.1 MySQL

**Purpose:** A relational database management system used to store, structure, and query your project data.

**Industry Usage:** One of the most widely used open-source databases in the world, powering everything from startups to large-scale web applications.

**Why It Is Used Here:** It is the database engine underlying your SQL project — every table, view, and procedure you built lives inside MySQL.

### 5.2 MySQL Workbench

**Purpose:** A graphical tool for designing schemas, writing SQL, and administering MySQL databases.

**Industry Usage:** Widely used by database administrators and developers for ER modelling, query development, and performance tuning.

**Why It Is Used Here:** It is where you created your ER diagram, built your tables, and tested your SQL scripts before connecting Python.

### 5.3 Python

**Purpose:** A general-purpose programming language, dominant in data analytics due to its readability and rich ecosystem of libraries.

**Industry Usage:** The primary language for data analysis, data engineering, automation, and machine learning across nearly every industry.

**Why It Is Used Here:** It is the language used to connect to your database, clean data, calculate KPIs, and generate visualizations.

### 5.4 Anaconda

**Purpose:** A Python distribution that bundles Python with hundreds of data-science packages and environment management tools.

**Industry Usage:** A common starting point for data analysts and scientists who need a pre-configured environment without manual setup.

**Why It Is Used Here:** It provides a ready-to-use environment containing Pandas, NumPy, Matplotlib, and Jupyter without individual installs.

## 5.5 Jupyter Notebook

**Purpose:** An interactive coding environment that combines code, output, and narrative text in a single document.

**Industry Usage:** The standard tool for exploratory analysis, prototyping, and sharing analysis walkthroughs with reasoning attached.

**Why It Is Used Here:** You will use it to explore your data step by step, view intermediate results, and document your reasoning inline.

## 5.6 Pandas

**Purpose:** A Python library providing the DataFrame — a table-like structure for cleaning, transforming, and analysing data.

**Industry Usage:** The single most-used data analysis library in Python; nearly every analytics workflow depends on it.

**Why It Is Used Here:** It is the tool you will use to load SQL query results, clean them, and compute your KPIs.

## 5.7 NumPy

**Purpose:** A library for fast numerical computation, underpinning array operations and mathematical functions.

**Industry Usage:** Used wherever performance-critical numerical work is required; it is the computational foundation beneath Pandas itself.

**Why It Is Used Here:** You will use it for numerical calculations such as averages, standard deviations, and array-based operations.

## 5.8 Matplotlib

**Purpose:** A plotting library used to create static charts — bar charts, line charts, pie charts, histograms, and more.

**Industry Usage:** The foundational visualization library in Python; many other libraries (like Seaborn) are built on top of it.

**Why It Is Used Here:** You will use it to turn your KPIs and trends into charts that support your business insights.

## 5.9 CSV Files

**Purpose:** A simple, universal file format for storing tabular data as plain text.

**Industry Usage:** A common intermediate and exchange format between systems, tools, and teams.

**Why It Is Used Here:** You will export cleaned data and KPI summaries to CSV for reporting, backup, and easy sharing.

## 5.10 Git

**Purpose:** A version control system that tracks changes to files over time and allows safe collaboration.

**Industry Usage:** The standard tool for managing source code changes in virtually every software and data team.

**Why It Is Used Here:** It tracks the history of your project files and protects your work as it evolves.

## 5.11 GitHub

**Purpose:** A cloud platform for hosting Git repositories, enabling sharing, showcasing, and collaboration.

**Industry Usage:** The most common platform recruiters check to review a candidate's real, hands-on work.

**Why It Is Used Here:** It is where you will publish your complete, organised project as your portfolio piece.

## 5.12 Stack Summary

| Layer                     | Tool(s)                    |
|---------------------------|----------------------------|
| Data Storage              | MySQL                      |
| Database Design & Admin   | MySQL Workbench            |
| Programming Language      | Python                     |
| Environment               | Anaconda, Jupyter Notebook |
| Data Analysis             | Pandas, NumPy              |
| Visualization             | Matplotlib                 |
| Data Exchange             | CSV                        |
| Version Control & Hosting | Git, GitHub                |

### NOTE

You do not need to master every tool before starting. This stack is introduced here so that when each tool appears in later chapters, you already know its role in the bigger picture.



---

## Chapter Summary

- The project uses a focused, industry-standard stack: MySQL, Python, Pandas, NumPy, Matplotlib, Git, and GitHub.
- Each tool has a specific, non-overlapping role in the workflow — from storage to analysis to publication.
- Anaconda and Jupyter Notebook provide the environment in which the Python phase is carried out.

## Key Takeaways

- MySQL and MySQL Workbench belong to the SQL phase you have already completed.
- Pandas and NumPy are the core analysis engines; Matplotlib is the core visualization engine.
- Git and GitHub turn your finished project into a shareable, professional portfolio piece.

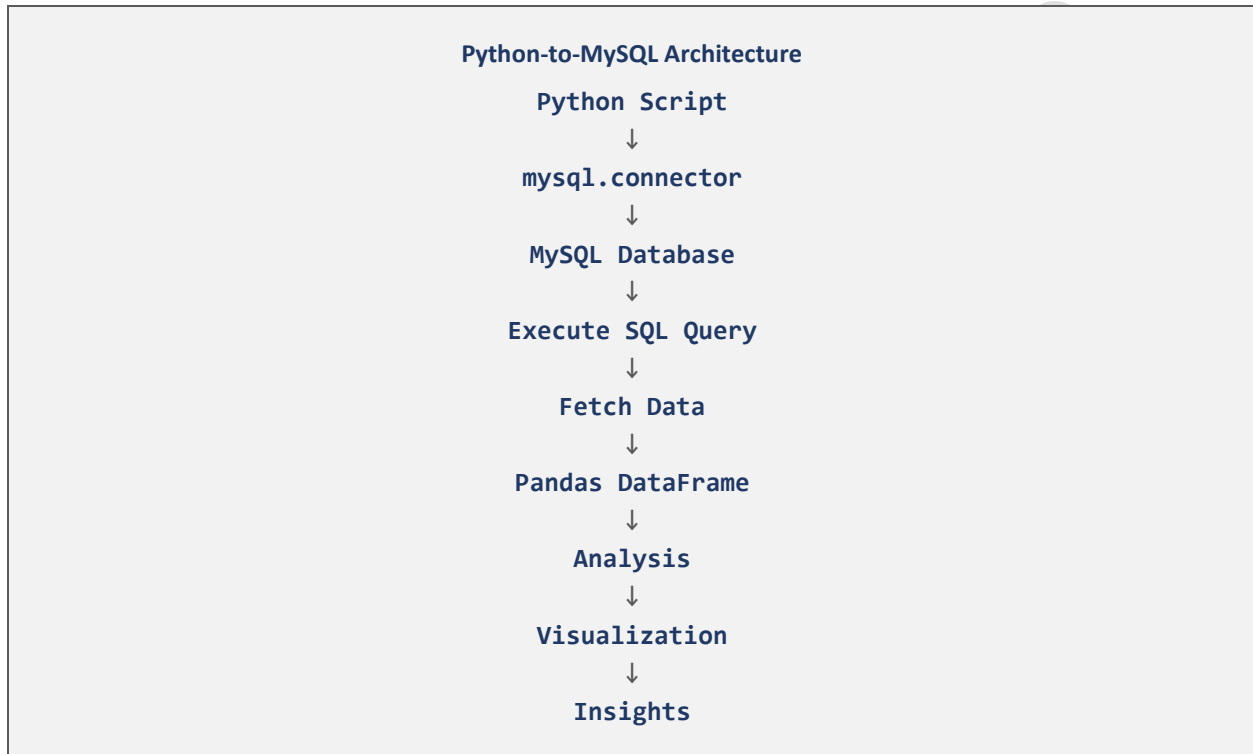
## Next Steps

- Confirm Anaconda, Jupyter Notebook, and MySQL Connector for Python are installed on your machine.
- Proceed to Chapter 6 to connect Python to your MySQL database for the first time.

## Chapter 6: Connecting Python with SQL

This chapter is the technical bridge between the two halves of your project. You will learn the architecture, the exact steps, and the working code needed to pull data from your MySQL database into Python for analysis.

### 6.1 Architecture Overview



Python does not read your database directly. It communicates through a connector library, which translates Python commands into the MySQL protocol, sends your SQL query, and returns the results as rows that Python can work with. Pandas then wraps those rows into a DataFrame — the table-like structure used for the rest of the analysis.

### 6.2 Installing the Connector

```
pip install mysql-connector-python pandas
```

Run this once in your Anaconda or terminal environment. If you are using Anaconda Navigator, you can alternatively install these packages through the Environments tab.

## 6.3 Establishing the Connection

```
import mysql.connector

connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password",
    database="your_database_name"
)

print("Connection successful:", connection.is_connected())
```

- host — the server address; "localhost" for a database running on your own machine.
- user and password — your MySQL credentials.
- database — the exact schema name you created in your SQL phase.

## 6.4 Reading an Entire Table

```
import pandas as pd

df = pd.read_sql("SELECT * FROM patients", connection)
print(df.head())
```

## 6.5 Reading a Custom SQL Query

In practice, you will rarely pull an entire table. Instead, write a targeted query — using joins, filters, and aggregations — so Python receives exactly the data it needs.

```
query = """
    SELECT d.department_name, COUNT(p.patient_id) AS total_patients
    FROM patients p
    JOIN departments d ON p.department_id = d.department_id
    GROUP BY d.department_name
    ORDER BY total_patients DESC
    """

df_dept = pd.read_sql(query, connection)
print(df_dept)
```

## 6.6 pandas.read\_sql() in Detail

`pandas.read_sql()` accepts either a raw SQL string or a SQLAlchemy connection object, executes it, and returns a fully-formed DataFrame with column names taken directly from your SELECT statement. This single function is the primary bridge used throughout the rest of this handbook.

| Parameter                | Purpose                                                   |
|--------------------------|-----------------------------------------------------------|
| <code>sql</code>         | The SQL query or table name to read                       |
| <code>con</code>         | The active database connection object                     |
| <code>index_col</code>   | Optional — column(s) to use as the DataFrame index        |
| <code>parse_dates</code> | Optional — column(s) to automatically convert to datetime |

## 6.7 Closing the Connection

```
connection.close()
print("Connection closed.")
```

### NOTE

Always close your connection once your data has been loaded into a DataFrame. Once the DataFrame exists, all further analysis happens in Python's memory — you no longer need the live database connection.

## 6.8 Common Errors and Troubleshooting

| Error                                | Likely Cause                                    | Fix                                              |
|--------------------------------------|-------------------------------------------------|--------------------------------------------------|
| Access denied for user               | Wrong username or password                      | Re-check MySQL credentials                       |
| Unknown database                     | Database name typed incorrectly                 | Confirm exact schema name in MySQL Workbench     |
| Can't connect to MySQL server        | MySQL service not running                       | Start the MySQL server / service                 |
| ModuleNotFoundError: mysql.connector | Package not installed in the active environment | Re-run pip install in the correct environment    |
| Empty DataFrame returned             | Query filters too strict, or table has no data  | Test the query directly in MySQL Workbench first |

### Best Practices

- Test every query in MySQL Workbench before running it from Python.
- Never hardcode passwords in a script you plan to publish on GitHub — use environment variables or a config file excluded via `.gitignore`.

- Write SQL queries that already filter and aggregate what you need, rather than pulling raw tables and processing everything in Pandas.
- Wrap your connection logic in a try/except block so failures are reported clearly instead of crashing silently.

**COMMON MISTAKE**

A common beginner mistake is leaving the database password directly in the script and pushing it to a public GitHub repository. Always remove credentials before committing, and use a .env file or config file instead.

**TIP**

Write one small reusable function, `get_connection()`, that returns a live connection object. Reuse it across every script in your project instead of repeating connection code.

## Chapter Summary

- Python connects to MySQL using the `mysql-connector-python` library, and `pandas.read_sql()` converts query results into DataFrames.
- Well-designed SQL queries reduce how much cleaning is needed once the data reaches Python.
- Connections should always be closed once data has been loaded, and credentials should never be committed to GitHub.

## Key Takeaways

- The connection architecture is: Python → connector → MySQL → query execution → DataFrame.
- `pandas.read_sql()` is the single most important function bridging your two project phases.
- Most connection errors trace back to credentials, database names, or the MySQL service not running.

## Next Steps

- Successfully connect to your own database and print the first five rows of one table.
- Proceed to Chapter 7 to see how your existing Python knowledge applies throughout this project.

## Chapter 7: Python Concepts Used in the Project

You do not need any new programming language for this project — you need to apply the Python you already know to a real dataset. This chapter maps every core Python and Pandas concept from your coursework to a concrete role inside this project.

### 7.1 Core Python Fundamentals

| Concept                | Where It Is Used                                   | Industry Example                                                                                                  |
|------------------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Variables              | Storing connection details, file paths, thresholds | Storing a KPI target value to compare against actuals                                                             |
| Functions              | Wrapping reusable logic (connect, clean, plot)     | A <code>get_connection()</code> function reused across scripts                                                    |
| Loops                  | Iterating over columns, files, or chart categories | Generating one chart per department automatically                                                                 |
| Conditional Statements | Flagging outliers, categorising values             | Marking a transaction as 'high risk' above a threshold                                                            |
| Lists                  | Holding column names, file paths, chart titles     | A list of KPI names to calculate in sequence                                                                      |
| Tuples                 | Fixed pairs like (width, height) for chart sizing  | Passing <code>figsize=(10, 6)</code> to a Matplotlib chart                                                        |
| Dictionary             | Mapping KPI names to values, or codes to labels    | <code>{'Cardiology': 'CARD', 'Neurology': 'NEURO'}</code>                                                         |
| Exception Handling     | Managing connection failures, bad data             | <code>try/except</code> around a database connection attempt                                                      |
| File Handling          | Reading/writing CSV, saving chart images           | Exporting a cleaned DataFrame to <code>cleaned_data.csv</code>                                                    |
| Modules                | Organising code into logical files                 | A separate <code>cleaning.py</code> module imported into <code>main.py</code>                                     |
| Packages               | Grouping related modules                           | A <code>utils</code> package containing <code>db</code> , <code>cleaning</code> , and <code>charts</code> modules |

## 7.2 NumPy in This Project

| Concept                 | Purpose                                | Where It Is Used                                 |
|-------------------------|----------------------------------------|--------------------------------------------------|
| Arrays                  | Fast, memory-efficient numeric storage | Underlying storage for numeric DataFrame columns |
| Mean / Median / Std Dev | Central tendency and spread            | Average order value, revenue variability         |
| Percentiles             | Understanding distribution shape       | Identifying the top 10% of customers by spend    |
| Vectorised Operations   | Fast column-wide calculations          | Calculating profit margin for every row at once  |

## 7.3 Pandas in This Project

| Concept     | Purpose                            | Where It Is Used                             |
|-------------|------------------------------------|----------------------------------------------|
| Filtering   | Selecting rows meeting a condition | Orders placed in the last 30 days            |
| Sorting     | Ordering rows by a column          | Top 10 products by revenue                   |
| Grouping    | Splitting data into categories     | Revenue grouped by department or region      |
| Aggregation | Summarising grouped data           | SUM, MEAN, COUNT applied after grouping      |
| Merge       | Combining DataFrames on a key      | Joining orders with customer details         |
| Join        | Combining DataFrames on the index  | Joining a KPI summary back onto a main table |

```
# Grouping and aggregation example
revenue_by_dept =
df.groupby("department_name")["bill_amount"].sum().sort_values(ascending=False)
print(revenue_by_dept)
```

## 7.4 Statistics Used in This Project

- Descriptive statistics: mean, median, mode, standard deviation, min/max — used to summarise KPIs.
- Distribution checks: histograms and value counts — used to understand how data is spread.
- Correlation: identifying whether two variables move together — for example, marketing spend vs. sales.
- Trend analysis: comparing values across time periods — for example, month-over-month growth.

## 7.5 Matplotlib in This Project

| Chart Type   | Best Used For                              | Example in This Project          |
|--------------|--------------------------------------------|----------------------------------|
| Bar Chart    | Comparing categories                       | Revenue by department            |
| Line Chart   | Trends over time                           | Monthly sales trend              |
| Pie Chart    | Share of a whole (few categories only)     | Market share by product category |
| Histogram    | Distribution of a single numeric variable  | Distribution of patient ages     |
| Scatter Plot | Relationship between two numeric variables | Marketing spend vs. revenue      |

## 7.6 Business KPI Calculations

KPI calculation combines everything above: filtering and grouping the right rows with Pandas, computing the right statistic with NumPy or Pandas, and formatting the result for reporting.

```
total_revenue = df["bill_amount"].sum()
avg_bill = df["bill_amount"].mean()
growth_rate = (this_month_revenue - last_month_revenue) / last_month_revenue * 100

print(f"Total Revenue: {total_revenue:.2f}")
print(f"Average Bill: {avg_bill:.2f}")
print(f"Growth Rate: {growth_rate:.2f}%")
```

### INDUSTRY INSIGHT

Interviewers frequently ask candidates to explain a `groupby().agg()` line or a KPI formula from their own project, live, on the spot. Make sure you can explain — in plain language — exactly what every line of your code does and why.

### NOTE

Chapters 7 and 8 are intentionally complementary: this chapter maps individual Python concepts to their use; Chapter 8 shows how they combine into one continuous analytics workflow.

## Chapter Summary

- Every core Python concept from your coursework has a direct, practical role in this project.
- Pandas provides the filtering, sorting, grouping, and merging tools used throughout the analysis phase.
- NumPy and Pandas together provide the statistics behind every KPI you will calculate.



## Key Takeaways

- Functions and modules keep your project code reusable and organised, not just "working".
- Exception handling protects your scripts from crashing on bad data or connection failures.
- Choosing the right chart type is as important as generating the chart itself.

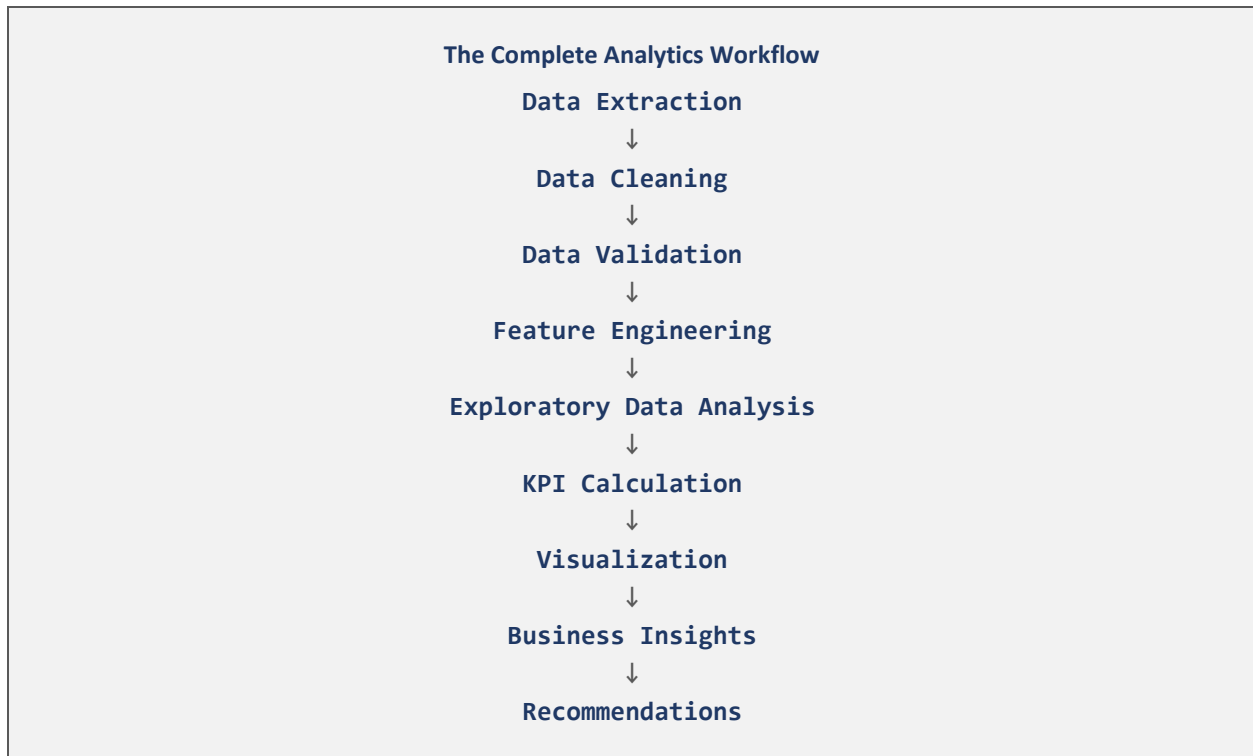
## Next Steps

- List the 4–6 KPIs you plan to calculate for your own domain, and note which Pandas operations each will need.
- Proceed to Chapter 8 to see the complete, combined analytics workflow in action.

© Shalinee Priya

## Chapter 8: Complete Analytics Workflow

This chapter walks through the full analytics pipeline end-to-end, using a single worked example (Hospital domain) so you can see exactly how each stage connects to the next. Apply the same pattern to your own domain and dataset.



### 8.1 Data Extraction

Extraction means pulling exactly the data you need from MySQL into a Pandas DataFrame, as covered in Chapter 6. Start broad enough to explore, but avoid pulling unrelated tables you will not use.

```
query = "SELECT * FROM patients"
df = pd.read_sql(query, connection)
```

### 8.2 Data Cleaning

Cleaning prepares raw data for reliable analysis. This is usually the most time-consuming stage of the entire workflow, and it directly determines whether your later insights can be trusted.

- Check for missing values with `df.isnull().sum()` and decide whether to fill, drop, or flag them.
- Check for duplicate rows with `df.duplicated().sum()` and remove genuine duplicates.
- Verify data types with `df.dtypes` — dates and numbers imported as text will silently break calculations.

- Standardise text values (e.g., "Male", "male", "M" should become one consistent value).

```
df.isnull().sum()
df = df.drop_duplicates()
df["admission_date"] = pd.to_datetime(df["admission_date"])
df["gender"] = df["gender"].str.strip().str.title()
```

### 8.3 Data Validation

Validation confirms the cleaned data is logically correct, not just technically well-formatted. This is where you sanity-check the data against real-world expectations.

- Are all foreign key values (e.g., department\_id) present in the related table?
- Are numeric values within a believable range (e.g., no negative ages, no bill amounts of zero for completed treatments)?
- Do date ranges make sense (e.g., discharge date is never before admission date)?

### 8.4 Feature Engineering

Feature engineering means creating new, more useful columns from the ones you already have. It is the step that turns raw data into analysis-ready data.

```
df["length_of_stay"] = (df["discharge_date"] - df["admission_date"]).dt.days
df["age_group"] = pd.cut(df["age"], bins=[0, 18, 35, 60, 100],
                        labels=["Child", "Young Adult", "Adult", "Senior"])
```

### 8.5 Exploratory Data Analysis (EDA)

EDA is open-ended investigation — you are not proving a hypothesis yet, you are getting to know the data. Look at shape, summary statistics, distributions, and obvious patterns before deciding what to formally measure.

```
print(df.shape)
print(df.describe())
print(df["age_group"].value_counts())
```

### 8.6 KPI Calculation

Once you understand the data, calculate the specific metrics your business problem requires.

```
avg_stay = df["length_of_stay"].mean()
readmission_rate = (df["is_readmission"].sum() / len(df)) * 100
revenue_by_dept = df.groupby("department_name")["bill_amount"].sum()
```

## 8.7 Visualization

Turn your KPIs into charts that a non-technical stakeholder can interpret in seconds.

```
import matplotlib.pyplot as plt

revenue_by_dept.sort_values(ascending=False).plot(kind="bar", color="#2E5395")
plt.title("Revenue by Department")
plt.ylabel("Revenue")
plt.tight_layout()
plt.savefig("Output/Charts/revenue_by_department.png")
plt.show()
```

## 8.8 Business Insights

Every chart should be followed by a short written insight using the Observation → Reason → Impact → Recommendation structure introduced in Chapter 11.

*Example: Cardiology generates the highest departmental revenue, likely due to higher-cost procedures and longer average stays. This makes Cardiology a priority area for resource planning and staff allocation.*

## 8.9 Recommendations

- Recommendations must be specific and actionable, not generic ("increase revenue" is not a recommendation).
- Tie every recommendation back to the KPI or chart that supports it.
- Where possible, quantify the expected impact of following the recommendation.

### INDUSTRY INSIGHT

In real analytics teams, a finished chart without a written insight is considered an unfinished deliverable. The chart shows what happened; the insight explains why it matters and what to do next.

### TIP

Keep a single running script or notebook per domain-area (e.g., revenue\_analysis.py) that performs extraction through visualization in one place. It makes your project far easier to explain and demo.

## Chapter Summary

- The complete analytics workflow has nine connected stages, from extraction to recommendations.
- Cleaning and validation are separate, sequential checks — one fixes format, the other checks logic.
- Every chart should be paired with a written business insight and, where possible, a recommendation.

## Key Takeaways

- Feature engineering (like `length_of_stay` or `age_group`) often unlocks the most valuable insights.
- KPIs should be calculated only after EDA has built a genuine understanding of the data.
- A chart without an insight is an unfinished deliverable in a professional setting.

## Next Steps

- Run this exact nine-stage workflow on one analysis area of your own domain, start to finish.
- Proceed to Chapter 9 to organise your project files professionally.

## Chapter 9: Project Directory Structure

A clean, predictable folder structure is one of the fastest ways to signal professionalism to anyone reviewing your project — a mentor, a recruiter, or your future self six months from now. This chapter defines the standard structure for your project folder.

### 9.1 Recommended Folder Structure

```
Project_Name/
├── README.md
├── requirements.txt
├── SQL/
│   ├── schema.sql
│   ├── sample_data.sql
│   └── advanced_queries.sql
├── Python/
│   ├── db_connection.py
│   ├── data_cleaning.py
│   ├── eda.py
│   ├── kpi_analysis.py
│   └── visualization.py
├── Data/
│   ├── raw/
│   └── cleaned/
├── Output/
│   ├── Charts/
│   └── Results/
├── Reports/
│   └── project_report.pdf
├── Documentation/
│   └── er_diagram.png
├── Presentation/
│   └── project_slides.pptx
├── Images/
│   └── screenshots/
```

### 9.2 What Each Folder Contains

| Folder / File    | Purpose                                                                  |
|------------------|--------------------------------------------------------------------------|
| README.md        | The front door of your project — overview, setup, and usage instructions |
| requirements.txt | List of Python packages needed to run the project                        |
| SQL/             | All SQL scripts: schema creation, sample data, advanced queries          |
| Python/          | All Python scripts, organised by workflow stage                          |
| Data/raw         | Unmodified data exported from the database, kept as a reference          |
| Data/cleaned     | Cleaned, analysis-ready data saved as CSV                                |

| Folder / File  | Purpose                                            |
|----------------|----------------------------------------------------|
| Output/Charts  | All generated chart images                         |
| Output/Results | KPI summaries and other generated result files     |
| Reports/       | The final written project report                   |
| Documentation/ | ER diagrams, schema diagrams, and design documents |
| Presentation/  | Slide deck used for your project demonstration     |
| Images/        | Screenshots used inside README or reports          |

### 9.3 Naming Conventions

- Use lowercase with underscores for file names: `kpi_analysis.py`, not `KPI Analysis.py`.
- Name scripts after what they do, not the order you wrote them in — avoid `script1.py`, `final_final.py`.
- Keep chart file names descriptive and consistent: `revenue_by_department.png`, not `chart1.png`.
- Use a single top-level folder named after your project, not "New Folder" or "Untitled".

#### TIP

Set up this exact folder structure on Day 1, even with empty placeholder files. It is far easier to fill in an organised structure than to reorganise a messy one right before submission.

#### NOTE

`requirements.txt` should list only the packages your project actually imports (e.g., `pandas`, `numpy`, `matplotlib`, `mysql-connector-python`), with versions if possible, so anyone can recreate your environment with one command: `pip install -r requirements.txt`.

## Chapter Summary

- A standard folder structure separates SQL, Python, data, output, documentation, and presentation materials.
- Consistent naming conventions make a project easy to navigate for anyone reviewing it.
- `requirements.txt` makes your project's environment reproducible by anyone who downloads it.

## Key Takeaways

- Raw and cleaned data should be kept in separate folders to preserve a clear before/after record.
- Charts and KPI results belong in `Output/`, separate from the scripts that generate them.
- Set up the full folder structure early — it is a planning tool, not just a final packaging step.

## Next Steps

- Create this folder structure for your own project right now, even if most folders start empty.
- Proceed to Chapter 10 to prepare this structure for a professional GitHub repository.

© Shalinee Priya



## Chapter 10: GitHub Repository Structure

Your GitHub repository is often the first thing a recruiter, mentor, or evaluator sees. This chapter explains how to package the folder structure from Chapter 9 into a professional, well-documented public repository.

### 10.1 Repository Naming

- Use a clear, descriptive, hyphenated name: hospital-management-data-analytics, not project1 or FinalProject.
- Avoid special characters, spaces, or ambiguous abbreviations in the repository name.
- The name should make the project's domain and purpose obvious at a glance.

### 10.2 Folder Organisation on GitHub

Your GitHub repository should mirror the local folder structure from Chapter 9 exactly — SQL/, Python/, Data/, Output/, Reports/, Documentation/, Presentation/, and Images/ at the top level, alongside README.md and requirements.txt.

### 10.3 Images and Screenshots

- Include screenshots of key charts, the ER diagram, and sample terminal/notebook output inside Images/.
- Embed 3–5 of the most important images directly in your README so visitors don't need to open the repository to understand the project.
- Compress large images before committing to keep the repository lightweight.

### 10.4 Recommended Commit Messages

Commit messages should describe what changed and why, in the present tense, so anyone reading your commit history understands the project's evolution.

| Good Commit Message                                 | Poor Commit Message |
|-----------------------------------------------------|---------------------|
| Add data cleaning script for patient records        | update              |
| Fix incorrect date parsing in admission_date column | fix bug             |
| Add revenue-by-department bar chart and insight     | changes             |
| Add README with setup instructions                  | readme              |

## 10.5 Branching (Optional)

For a solo student project, working directly on the main branch is acceptable. If you want to practise industry workflow, create a feature branch for each major stage (e.g., feature/eda, feature/visualization) and merge it back into main once complete.

## 10.6 GitHub Best Practices

- Never commit database passwords or personal credentials — use a .gitignore file to exclude config files.
- Commit in small, logical increments rather than one giant final upload.
- Keep the README up to date as the single source of truth for anyone visiting the repository.
- Pin or link the final project report and presentation from the README for easy access.

### COMMON MISTAKE

Uploading an entire project as a single commit titled 'final upload' hides your development process and looks unprofessional. Commit incrementally as you complete each phase of the roadmap in Chapter 3.

## 10.7 Professional README Template

```
# Project Title

End-to-End Data Analytics Project – [Your Domain] (SQL + Python)

## Overview
A short paragraph describing the business problem this project solves
and the complete workflow from database design to business insights.

## Tech Stack
MySQL, Python, Pandas, NumPy, Matplotlib, Jupyter Notebook, Git, GitHub

## Project Structure
(Insert the folder tree from Chapter 9 here)

## Business Problem
Describe the real-world problem this project addresses.

## Database Design
Short description + link/image of your ER diagram.

## Key Analyses & KPIs
- KPI 1: description
- KPI 2: description
- KPI 3: description

## Sample Insights
```

```

- Insight 1
- Insight 2

## How to Run This Project
1. Clone this repository
2. Install dependencies: pip install -r requirements.txt
3. Import SQL/schema.sql and SQL/sample_data.sql into MySQL
4. Update database credentials in Python/db_connection.py
5. Run scripts in the Python/ folder in order

## Sample Charts
(Embed 2-3 chart images here)

## Author
Your Name – [LinkedIn/Portfolio link]

```

**TIP**

Treat your README as a short pitch, not a full report. It should let a visitor understand your project in under two minutes, with links to the full report and presentation for anyone who wants more depth.

---

## Chapter Summary

- A GitHub repository should mirror your local folder structure and include a clear, descriptive name.
- Commit messages should describe what changed and why, committed incrementally rather than all at once.
- A strong README acts as a two-minute pitch for the entire project.

## Key Takeaways

- Never commit credentials — use .gitignore to protect sensitive configuration files.
- Embedding key images directly in the README dramatically improves first impressions.
- Branching is optional for solo projects but is good practice to demonstrate awareness of team workflows.

## Next Steps

- Create your repository, add a .gitignore, and push your first incremental commit.
- Proceed to Chapter 11 to learn how to write the business insights your README references.

## Chapter 11: Business Insights

This is the chapter that separates a data project from a data analytics project. Anyone can produce a chart. Very few people can look at that chart and tell a business exactly what it means and what to do about it. This chapter teaches you that skill directly.

### 11.1 Why Charts Alone Are Not Enough

A chart shows what happened. It does not explain why it happened, whether it matters, or what should be done next. A stakeholder looking at a bar chart of "Revenue by Department" still has to ask three questions themselves: Why is this the pattern? Does it matter to the business? What should we do about it? A professional analyst answers all three questions before the stakeholder has to ask.

#### COMMON MISTAKE

Submitting a project that is a sequence of charts with captions like "This chart shows revenue by department" is one of the most common ways student projects lose marks and fail to impress reviewers. A caption describes; an insight explains and acts.

### 11.2 The Insight Framework

| The Insight-Writing Framework                                                                                                                          |                          |                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------------------------------------------|
| <p style="text-align: center;"> <b>Observation</b><br/> ↓<br/> <b>Reason</b><br/> ↓<br/> <b>Business Impact</b><br/> ↓<br/> <b>Recommendation</b> </p> |                          |                                                                           |
| Stage                                                                                                                                                  | Question It Answers      | Example                                                                   |
| Observation                                                                                                                                            | What does the data show? | Cardiology accounts for 32% of total hospital revenue.                    |
| Reason                                                                                                                                                 | Why is this happening?   | Cardiology procedures have higher average costs and longer stays.         |
| Business Impact                                                                                                                                        | Why does this matter?    | Any disruption to Cardiology capacity directly threatens overall revenue. |
| Recommendation                                                                                                                                         | What should be done?     | Prioritise staffing and equipment investment in Cardiology first.         |

## 11.3 Worked Examples Across Domains

### Example 1 — E-commerce

- Observation: Cart abandonment rate is 68% for orders above ₹5,000.
- Reason: High-value orders likely trigger hesitation around shipping cost or delivery time.
- Business Impact: The business is losing a disproportionate share of high-value revenue at checkout.
- Recommendation: Test offering free shipping or an EMI option specifically above the ₹5,000 threshold.

### Example 2 — Banking

- Observation: Loan default rate is 3x higher for loans issued without a co-applicant.
- Reason: Single-applicant loans may carry higher risk with less financial backup.
- Business Impact: Continuing current approval criteria increases exposure to bad debt.
- Recommendation: Introduce stricter income verification for single-applicant loan approvals.

### Example 3 — Food Delivery

- Observation: Average delivery time exceeds 45 minutes during 7–9 PM on weekends.
- Reason: Order volume peaks sharply in this window while rider availability stays flat.
- Business Impact: Late deliveries in this window likely drive most customer complaints and cancellations.
- Recommendation: Introduce a weekend evening rider incentive to increase availability during peak hours.

## 11.4 Writing Insights: Do's and Don'ts

| Do                                                             | Don't                                                |
|----------------------------------------------------------------|------------------------------------------------------|
| Tie every insight to a specific number from your analysis      | Write vague statements like "sales are good"         |
| Use plain business language a non-technical reader understands | Use technical jargon like "the p-value indicates..." |
| Keep each insight to 3–5 sentences                             | Write a full paragraph restating the chart in words  |
| Make recommendations specific and actionable                   | Write recommendations like "improve performance"     |
| Quantify impact where possible                                 | Leave business impact implied rather than stated     |

### INDUSTRY INSIGHT

In interviews, being asked to walk through one insight from your project — out loud, unscripted — is common. Practise saying your Observation → Reason → Impact → Recommendation for your top 3 insights until it feels natural, not memorised.

| Do                                                                                                                                                                                                                  | Don't |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <b>TIP</b><br>Write your insights as you go, immediately after each chart, while the context is fresh. Trying to write all insights at the end, after finishing every chart, produces weaker, more generic writing. |       |
| <b>NOTE</b><br>Not every chart needs a full four-part insight. Minor supporting charts can carry a single-sentence observation. Reserve the full framework for your 4–6 most important findings.                    |       |

## Chapter Summary

- A chart alone answers 'what happened'; a business insight answers 'why it matters and what to do'.
- The Observation → Reason → Business Impact → Recommendation framework structures every strong insight.
- Recommendations must be specific and actionable, not generic statements.

## Key Takeaways

- Insights should be written in plain business language, not technical jargon.
- Quantifying business impact makes an insight far more persuasive and credible.
- The strongest projects contain 4–6 fully developed insights rather than a caption under every chart.

## Next Steps

- Write the full four-part insight for your three most important charts.
- Proceed to Chapter 12 to check your insights against the complete final deliverables checklist.

## Chapter 12: Final Deliverables

---

This chapter provides a single, comprehensive checklist of everything your completed project should contain. Use it as a running tracker throughout the project, not only at the end.

### 12.1 SQL Phase Deliverables

- ☐ Business Problem statement
- ☐ Requirement Analysis document
- ☐ ER Diagram
- ☐ Database Schema (normalised to 3NF)
- ☐ SQL Scripts for table creation with constraints
- ☐ Advanced SQL Queries (joins, subqueries, aggregations)
- ☐ Views
- ☐ Stored Procedures
- ☐ Triggers
- ☐ Window Functions

### 12.2 Python Analytics Phase Deliverables

- ☐ Python source code, organised by workflow stage
- ☐ Working database connection script
- ☐ Data cleaning script(s) with clear before/after checks
- ☐ Exploratory Data Analysis (EDA) with documented findings
- ☐ Calculated business KPIs relevant to the chosen domain
- ☐ A complete set of charts covering the key business questions
- ☐ Business insights written using the Observation → Reason → Impact → Recommendation framework
- ☐ Clear, specific recommendations tied to each major insight

### 12.3 Documentation & Presentation Deliverables

- ☐ Project Report (business problem through recommendations)
- ☐ Presentation slide deck
- ☐ GitHub Repository, fully organised per Chapter 9 and Chapter 10
- ☐ README.md with setup instructions and embedded sample charts
- ☐ Project Demonstration (live or recorded walkthrough)

**NOTE**

This checklist is intentionally comprehensive. Chapter 15 provides a shorter, submission-day verification checklist once every item here has already been completed.

## Chapter Summary

- Final deliverables span three groups: SQL phase, Python analytics phase, and documentation/presentation.
- Every SQL deliverable from your original project remains part of this handbook's final submission.
- Documentation and presentation deliverables are treated as formal, required outputs, not optional extras.

## Key Takeaways

- Use this checklist continuously throughout the project, not only at the very end.
- Missing even one advanced SQL feature (views, procedures, triggers, window functions) is a common, avoidable gap.
- A project is only complete when all three deliverable groups are finished, not just the code.

## Next Steps

- Go through each checklist item now and mark your current completion status honestly.
- Proceed to Chapter 13 to review the best practices that make these deliverables genuinely professional.



## Chapter 13: Best Practices

These are the habits that distinguish a professional-looking project from a student assignment. None of them require advanced skill — they require consistency.

### 13.1 Professional Coding Standards

- Write code that could be understood by a teammate who has never seen your project before.
- Prefer clear, slightly longer variable names over short, cryptic ones.
- Keep functions focused on a single responsibility (one function, one job).
- Avoid duplicating the same logic in multiple scripts — extract it into a shared function instead.

### 13.2 Naming Conventions

| Item                   | Convention                                  | Example                           |
|------------------------|---------------------------------------------|-----------------------------------|
| Python files           | snake_case                                  | data_cleaning.py                  |
| Variables & functions  | snake_case                                  | avg_bill_amount, get_connection() |
| SQL tables & columns   | snake_case, singular or plural consistently | patient, department_id            |
| Chart files            | descriptive snake_case                      | revenue_by_department.png         |
| Git branches (if used) | type/short-description                      | feature/eda, fix/date-parsing     |

### 13.3 Documentation

- Add a short comment above any code block whose purpose isn't immediately obvious.
- Document assumptions directly in the code (e.g., "Assuming missing gender values represent 'Unknown'").
- Keep the README current — update it whenever the project structure or setup steps change.

### 13.4 Folder Organisation

Revisit Chapter 9's structure regularly during the project, not only at the start. A folder structure that drifts from the standard becomes harder to navigate with every new file.

### 13.5 SQL Formatting

```
-- Good formatting: keywords capitalised, one clause per line
SELECT d.department_name, COUNT(p.patient_id) AS total_patients
FROM patients p
JOIN departments d ON p.department_id = d.department_id
```

```
GROUP BY d.department_name
ORDER BY total_patients DESC;
```

- Capitalise SQL keywords (SELECT, FROM, WHERE, JOIN) for readability.
- Put each major clause on its own line for longer queries.
- Use meaningful table aliases (p for patients, d for departments), not single unrelated letters.

### 13.6 Python Formatting

- Follow PEP 8 conventions: 4-space indentation, lowercase snake\_case names, reasonable line lengths.
- Group imports at the top of each file: standard library, third-party, then local modules.
- Leave blank lines between logical sections of a script for readability.

### 13.7 Comments

Comments should explain why something is done, not restate what the code already makes obvious. "# remove rows with missing patient IDs since they cannot be linked to billing" is far more useful than "# drop na".

### 13.8 Exception Handling

```
try:
    connection = mysql.connector.connect(**config)
except mysql.connector.Error as e:
    print(f"Database connection failed: {e}")
    raise
```

### 13.9 Version Control

- Commit early and often, with meaningful messages (see Chapter 10.4).
- Never treat GitHub as a final dumping ground — use it as your working history.
- Use a .gitignore to exclude environment files, credentials, and unnecessary large files.

### 13.10 Code Reusability & Modular Programming

Break your project into modules by responsibility: a connection module, a cleaning module, an analysis module, and a visualization module. Each script should import what it needs rather than duplicating logic.

```
from db_connection import get_connection
from data_cleaning import clean_patient_data

connection = get_connection()
df = clean_patient_data(connection)
```

### 13.11 Project Backup

- Keep your project on GitHub as your primary backup, updated regularly, not just locally on one machine.
- Periodically export your database schema and data as a .sql backup file.
- Avoid relying on a single device — a lost laptop should never mean a lost project.

### 13.12 Professional Presentation Tips

- Open your presentation with the business problem, not the tech stack.
- Show your ER diagram briefly, then spend most of your time on insights and recommendations.
- Rehearse explaining at least one query and one KPI calculation clearly out loud.
- End with a short section on what you would improve or extend with more time.

#### TIP

Best practices are easiest to follow when adopted from Day 1. Retrofitting good structure and naming into a messy, near-finished project takes far longer than doing it right from the start.

---

## Chapter Summary

- Professional coding standards, naming conventions, and documentation apply equally to SQL and Python.
- Modular, reusable code is easier to maintain, explain, and extend.
- Version control and regular backups protect your project from loss and demonstrate professional discipline.

## Key Takeaways

- Consistent naming (Section 13.2) is a small habit with an outsized impact on how professional a project looks.
- Comments should explain reasoning, not restate obvious code.
- A strong presentation leads with the business problem and closes with insights, not with a tour of the tech stack.

## Next Steps

- Audit your current code and folders against Sections 13.1–13.4 and fix any inconsistencies now.
- Proceed to Chapter 14 to learn the specific mistakes that most commonly undermine student projects.

## Chapter 14: Common Mistakes

This chapter lists the mistakes seen most frequently in student analytics projects, alongside how to avoid each one. Reviewing this list before you submit is one of the highest-value things you can do with a spare hour.

| Common Mistake           | How to Avoid It                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Poor Database Design     | Model entities and relationships carefully with an ER diagram before creating any tables; don't design tables directly in code. |
| Improper Normalization   | Check each table against 1NF, 2NF, and 3NF rules; remove repeating groups and partial or transitive dependencies.               |
| Duplicate Data           | Enforce primary keys and unique constraints at the database level; run <code>df.duplicated().sum()</code> during cleaning.      |
| Hardcoded Values         | Use variables, config files, or database lookups instead of typing fixed values directly into queries or scripts.               |
| Missing Constraints      | Add NOT NULL, FOREIGN KEY, UNIQUE, and CHECK constraints wherever business rules require them.                                  |
| Improper SQL Queries     | Test every query in MySQL Workbench first; verify row counts and sample output before using it in Python.                       |
| No Exception Handling    | Wrap database connections and file operations in try/except blocks with clear error messages.                                   |
| Poor Folder Organisation | Follow the standard structure in Chapter 9 from the start of the project, not as a last-minute cleanup.                         |
| Weak Documentation       | Keep the README updated continuously; document assumptions and setup steps as you make them.                                    |
| No Business Insights     | Apply the Observation → Reason → Impact → Recommendation framework from Chapter 11 to every major chart.                        |
| Wrong Chart Selection    | Match the chart type to the question being asked, using the guide in Section 7.5.                                               |
| Poor README              | Use the professional README template from Chapter 10.7 and embed sample charts and results.                                     |

### NOTE

If you recognise your project in more than two or three rows of this table, treat it as a priority fix list — these are also the issues reviewers and interviewers notice first.

## Chapter Summary

- Most common student mistakes trace back to skipped planning steps: ER diagrams, normalization checks, or documentation habits.
- Technical mistakes (missing constraints, no exception handling) and communication mistakes (no insights, weak README) are equally damaging.
- Every mistake in this chapter maps directly to a fix already covered earlier in this handbook.

## Key Takeaways

- Wrong chart selection and missing business insights are analytics-specific mistakes, distinct from database mistakes.
- A poor README can undermine an otherwise strong project by hiding its quality from the reader.
- Reviewing this chapter before submission is a fast, high-value final check.

## Next Steps

- Go through this table line by line against your own project and note any matches honestly.
- Proceed to Chapter 15 for the final, submission-day checklist.

## Chapter 15: Final Submission Checklist

---

This is your last stop before submission. Work through every item below in order. Do not submit until every box can be honestly checked.

### 15.1 Database & SQL

- ☐ Business problem statement is clear and specific
- ☐ ER diagram matches the final database schema exactly
- ☐ Schema is normalised to at least 3NF
- ☐ All tables have appropriate primary keys, foreign keys, and constraints
- ☐ Sample data is realistic and sufficient for meaningful analysis
- ☐ Views, stored procedures, triggers, and window functions are implemented and tested

### 15.2 Python & Analysis

- ☐ Python successfully connects to the database using credentials that are not hardcoded in the committed script
- ☐ Data is cleaned, with missing values, duplicates, and data types explicitly handled
- ☐ Data validation checks have been performed and documented
- ☐ EDA has been carried out with clear, documented findings
- ☐ All planned KPIs have been calculated correctly
- ☐ Charts cover the key business questions and use appropriate chart types
- ☐ Every major chart is paired with a written business insight and recommendation

### 15.3 Documentation & Repository

- ☐ Project report is complete, from business problem through recommendations
- ☐ README.md follows the professional template and includes sample charts
- ☐ requirements.txt lists all necessary Python packages
- ☐ Folder structure matches the standard defined in Chapter 9
- ☐ GitHub repository is public/shared, with incremental, descriptive commit history
- ☐ No credentials, passwords, or sensitive data are present anywhere in the repository

### 15.4 Presentation & Demonstration

- ☐ Slide deck opens with the business problem, not the tech stack
- ☐ ER diagram and database design are covered briefly, not in excessive detail

- ☐ At least 3 fully developed business insights are ready to present confidently
- ☐ You can explain, out loud, at least one SQL query and one Python KPI calculation line by line
- ☐ A closing section covers potential improvements or extensions
- ☐ The presentation has been rehearsed at least once, start to finish, within the time limit

**NOTE**

Congratulations on reaching the final checklist. If every box above is checked honestly, your End-to-End Data Analytics Project meets a professional, industry-aligned standard — not just an academic one. Good luck with your submission and presentation.